

UIL COMPUTER SCIENCE WRITTEN TEST

2023 INVITATIONAL B

FEBRUARY/MARCH 2023

General Directions (Please read carefully!)

1. DO NOT OPEN THE EXAM UNTIL TOLD TO DO SO.
2. There are 40 questions on this contest exam. You will have 45 minutes to complete this contest.
3. All answers must be legibly written on the answer sheet provided. Indicate your answers in the appropriate blanks provided on the answer sheet. Clean erasures are necessary for accurate grading.
4. You may write on the test packet or any additional scratch paper provided by the contest director, but NOT on the answer sheet, which is reserved for answers only.
5. All questions have ONE and only ONE correct answer. There is a 2-point penalty for all incorrect answers.
6. Tests may not be turned in until 45 minutes have elapsed. If you finish the test before the end of the allotted time, remain at your seat and retain your test until told to do otherwise. You may use this time to check your answers.
7. If you are in the process of actually writing an answer when the signal to stop is given, you may finish writing that answer.
8. All provided code segments are intended to be syntactically correct, unless otherwise stated. You may also assume that any undefined variables are defined as used.
9. A reference to many commonly used Java classes is provided with the test, and you may use this reference sheet during the contest. AFTER THE CONTEST BEGINS, you may detach the reference sheet from the test booklet if you wish.
10. Assume that any necessary import statements for standard Java SE packages and classes (e.g., `java.util`, `System`, etc.) are included in any programs or code segments that refer to methods from these classes and packages.
11. NO CALCULATORS of any kind may be used during this contest.

Scoring

1. Correct answers will receive **6 points**.
2. Incorrect answers will lose **2 points**.
3. Unanswered questions will neither receive nor lose any points.
4. In the event of a tie, the student with the highest percentage of attempted questions correct shall win the tie.

STANDARD CLASSES AND INTERFACES – SUPPLEMENTAL REFERENCE

package java.lang

```
class Object
    boolean equals(Object anotherObject)
    String toString()
    int hashCode()

interface Comparable<T>
    int compareTo(T anotherObject)
        Returns a value < 0 if this is less than anotherObject.
        Returns a value = 0 if this is equal to anotherObject.
        Returns a value > 0 if this is greater than anotherObject.

class Integer implements Comparable<Integer>
    Integer(int value)
    int intValue()
    boolean equals(Object anotherObject)
    String toString()
    String toString(int i, int radix)
    int compareTo(Integer anotherInteger)
    static int parseInt(String s)

class Double implements Comparable<Double>
    Double(double value)
    double doubleValue()
    boolean equals(Object anotherObject)
    String toString()
    int compareTo(Double anotherDouble)
    static double parseDouble(String s)

class String implements Comparable<String>
    int compareTo(String anotherString)
    boolean equals(Object anotherObject)
    int length()
    String substring(int begin)
        Returns substring(begin, length()).
    String substring(int begin, int end)
        Returns the substring from index begin through index (end - 1).
    int indexOf(String str)
        Returns the index within this string of the first occurrence of str. Returns
        -1 if str is not found.
    int indexOf(String str, int fromIndex)
        Returns the index within this string of the first occurrence of str, starting
        the search at fromIndex. Returns -1 if str is not found.
    int indexOf(int ch)
    int indexOf(int ch, int fromIndex)
    char charAt(int index)
    String toLowerCase()
    String toUpperCase()
    String[] split(String regex)
    boolean matches(String regex)
    String replaceAll(String regex, String str)

class Character
    static boolean isDigit(char ch)
    static boolean isLetter(char ch)
    static boolean isLetterOrDigit(char ch)
    static boolean isLowerCase(char ch)
    static boolean isUpperCase(char ch)
    static char toUpperCase(char ch)
    static char toLowerCase(char ch)

class Math
    static int abs(int a)
    static double abs(double a)
    static double pow(double base, double exponent)
    static double sqrt(double a)
    static double ceil(double a)
    static double floor(double a)
    static double min(double a, double b)
    static double max(double a, double b)
    static int min(int a, int b)
    static int max(int a, int b)
    static long round(double a)
    static double random()
        Returns a double greater than or equal to 0.0 and less than 1.0.
```

package java.util

```
interface List<E>
class ArrayList<E> implements List<E>
    boolean add(E item)
    int size()
    Iterator<E> iterator()
    ListIterator<E> listIterator()
    E get(int index)
    E set(int index, E item)
    void add(int index, E item)
    E remove(int index)

class LinkedList<E> implements List<E>, Queue<E>
    void addFirst(E item)
    void addLast(E item)
    E getFirst()
    E getLast()
    E removeFirst()
    E removeLast()

class Stack<E>
    boolean isEmpty()
    E peek()
    E pop()
    E push(E item)

interface Queue<E>
class PriorityQueue<E>
    boolean add(E item)
    boolean isEmpty()
    E peek()
    E remove()

interface Set<E>
class HashSet<E> implements Set<E>
class TreeSet<E> implements Set<E>
    boolean add(E item)
    boolean contains(Object item)
    boolean remove(Object item)
    int size()
    Iterator<E> iterator()
    boolean addAll(Collection<? extends E> c)
    boolean removeAll(Collection<?> c)
    boolean retainAll(Collection<?> c)

interface Map<K,V>
class HashMap<K,V> implements Map<K,V>
class TreeMap<K,V> implements Map<K,V>
    Object put(K key, V value)
    V get(Object key)
    boolean containsKey(Object key)
    int size()
    Set<K> keySet()
    Set<Map.Entry<K, V>> entrySet()

interface Iterator<E>
    boolean hasNext()
    E next()
    void remove()

interface ListIterator<E> extends Iterator<E>
    void add(E item)
    void set(E item)

class Scanner
    Scanner(InputStream source)
    Scanner(String str)
    boolean hasNext()
    boolean hasNextInt()
    boolean hasNextDouble()
    String next()
    int nextInt()
    double nextDouble()
    String nextLine()
    Scanner useDelimiter(String regex)
```

STANDARD CLASSES AND INTERFACES – SUPPLEMENTAL REFERENCE

Package <code>java.util.function</code>		
<pre>Interface BiConsumer<T,U> void accept(T t, U u) Interface BiFunction<T,U,R> R apply(T t, U u) Interface BiPredicate<T,U> boolean test(T t, U u) Interface Consumer<T> void accept(T t) Interface Function<T,R> R apply(T t) Interface Predicate<T> boolean test(T t) Interface Supplier<T> T get()</pre>		

UIL COMPUTER SCIENCE WRITTEN TEST – 2023 INVITATIONAL B

Note: Correct responses are based on **Java SE Development Kit 17 (JDK 17)** from Oracle, Inc. All provided code segments are intended to be syntactically correct, unless otherwise stated (e.g., "error" is an answer choice) and any necessary Java SE 17 Standard Packages have been imported. Ignore any typographical errors and assume any undefined variables are defined as used. **For all output statements, assume that the System class has been statically imported using: `import static java.lang.System.*;`**

Question 1

What is the sum of 10010111_2 and 10100011_2 ?

- A) 1410_{16} B) DA_{16} C) $13A_{16}$ D) $19A_{16}$ E) 1310_{16}

Question 2

What is the output of the code segment to the right?

- A) 3 B) 4 C) 5 D) 6 E) 6.25

```
out.print(1 + 29 % 10 / 4 + 3);
```

Question 3

What is the output of the code segment to the right?

- A) Blue
Red
GreenYellow
PurpleOrange

- B) Blue
Red
Green
Yellow
PurpleOrange

- C) Blue
Red
Green
YellowPurple
Orange

- D) BlueRed
GreenYellowPurpleOrange

- E) Blue
Red
GreenYellowPurple
Orange

```
out.print("Blue\nRed\nGreen");
out.print("Yellow\nPurple");
out.print("Orange");
```

Question 4

What is the output of the code segment to the right?

- A) 1 B) 2 C) 7 D) 8 E) 9

```
String str = "Scholastic";
out.print(str.indexOf("c", 2));
```

Question 5

What is the output of the code segment to the right?

- A) true
B) false

```
boolean A = true;
boolean B = !A;
out.print(!A && (A || B) ^ B);
```

Question 6

What is the output of the code segment to the right?

- A) 5 B) 5.0 C) 6 D) 6.0 E) 15

```
double M = Math.sqrt(31);
out.print((int) Math.floor(M));
```

Question 7 What is the output of the code segment to the right? A) 6.4 B) 3.2 C) 6.2 D) 6.0 E) 6	<pre>double T = 2.2; double A = 8 * 4 / 10 + 1 + T ; out.print(A);</pre>
Question 8 What is the output of the code segment to the right? A) B B) BCDEF C) BCDEFG D) F E) FG	<pre>int Q = 20 % 7; switch (Q) { case 1:out.print("A"); case 2:out.print("B"); case 3:out.print("C"); case 4:out.print("D"); case 5:out.print("E"); case 6:out.print("F"); default:out.print("G"); }</pre>
Question 9 How many x's will be output of the code segment to the right? A) 4 B) 5 C) 6 D) 7 E) 12	<pre>for(int x = 50; x > 2; x = x / 2 + 1) out.print("x");</pre>
Question 10 What is the output of the code segment to the right? A) 49 B) 45 C) 41 D) 40 E) 36	<pre>int[] four = {4,8,12,16,20,24,28}; int[] five = {5,10,15,20,25,30,35}; if (four.length<five.length) out.print(four[4]+five[5]); else out.print(four[5]+five[4]);</pre>
Question 11 What is output by the code segment to the right? A) 3684 B) 12243648 C) 361224 D) 363648 E) 3636	<pre>String St = "12 24 36 48"; Scanner go = new Scanner(St); out.print(go.nextInt() + go.nextInt()); out.print(go.next() + go.next());</pre>
Question 12 What is the output of the code segment to the right? A) 113 B) 123 C) 73 D) 197 E) 63	<pre>int sum = 0; for(int x = 1; x <= 50; x = x * 2) sum += x + 10; out.print(sum);</pre>
Question 13 What is the output of the code segment to the right? A) 11 B) 12 C) 13 D) 14 E) 15	<pre>int a = 15, b = 12, c = 9 ; out.print(c ^ b a - c & b ^ a);</pre>
Question 14 What is the output of the code segment shown on the right? A) 0 B) 1 C) -1 D) 127 E) -129	<pre>out.println(Byte.MIN_VALUE + 127);</pre>

Question 15 What is output by the code segment to the right? A) [12, 10, 14, 16, 10] B) [10, 12, 10, 14, 16, 10] C) [12, 12, 14, 16, 10] D) [10, 12, 12, 14, 16, 10] E) [10, 12, 14, 16, 10]	<pre>ArrayList<Integer> messi; messi = new ArrayList<Integer>(); messi.add(10); messi.add(12); messi.add(messi.get(1)); messi.add(14); messi.remove(0); messi.add(16); messi.add(10); messi.add(0,messi.get(0)-2); out.println(messi);</pre>
Question 16 What is the output of the code segment shown on the right? A) GJ B) IJ C) JB D) DE E) ED	<pre>String D = "ABCDEFGH IJ"; for(int x=1; x<=4; x++) D = D.substring(3)+D.substring(0,1); out.println(D);</pre>
Question 17 What is the output of the code segment shown on the right? A) 169 B) 144 C) 121 D) 100 E) 81	<pre>ArrayList<Integer>T; T = new ArrayList<Integer>(); T.add(3); T.add(5); for (int x=2;x<=100;x++) { int N = T.get(x-2) + T.get(x-1); T.add(N); } out.print(T.get(8));</pre>
Question 18 What is the output of the code segment shown on the right? A) 3 B) 6 C) 9 D) 12 E) 14	<pre>String St = ""; for(char ch = 'A'; ch<='C'; ch++) St += ch + St + ch; out.print(St.length());</pre>
Question 19 What is the output of the code segment shown on the right? A) 3 B) 4 C) 5 D) 6 E) 7	<pre>int[] G = {5,1,2,9,2,6,7,4,1,7}; for(int x=1; x<=8; x++) G[x] = (G[x-1] + G[x+1])/2; out.println(G[5]);</pre>

Question 20 In the code segment to the right, what is the output of line 1? A) 0 B) 1 C) 2 D) 3 E) 4	
Question 21 In the code segment to the right, what is the output of line 2? A) 0 B) 1 C) 2 D) 3 E) 4	
Question 22 In the code segment to the right, if the first line: <code>int[] cool = {17,19,12,8,3}</code> was changed to <code>int[] cool = {17,19,12,8,3,1,22,11,5,6}</code> What would be the final value of <code>list[0]</code>? A) 1 B) 3 C) 5 D) 7 E) 9	<pre>int[] cool = {17,19,12,8,3}; int[] list = new int[cool.length]; for(int x=0;x<cool.length;x++) for(int y=x+1;y<cool.length;y++) if(cool[x]>=cool[y]) list[x]++; else list[y]++; out.print(list[0]); //line 1 out.print(list[4]); //line 2</pre>
Question 23 What is the output of the code segment shown on the right? A) CAB B) BAC C) 6 D) 216 E) 198	<pre>String St = "CAB"; int N = St.charAt(2); N += St.charAt(1); N += St.charAt(0); out.print(N);</pre>
Question 24 In the code on the right, how many *s will be printed? A) 770 B) 28 C) 800 D) 700 E) 880	<pre>for(int A = 1; A <= 10; A++) for(int B = -5; B <= 5; B++) for(int C = 8; C > 1; C--) out.print("*");</pre>
Question 25 What is returned by the method call <code>Yes (5, 2)</code> A) 2 B) 5 C) 7 D) 10 E) 6	
Question 26 What is returned by the method call <code>Yes (3, 3)</code> A) 9 B) 24 C) 6 D) 12 E) 18	
Question 27 What is returned by the method call <code>Yes (0, 7)</code> A) 20 B) 24 C) 26 D) 28 E) 30	<pre>public static int Yes(int x, int y) { if (x>y) return x * y; if (x == y) return Yes(x+y,y+1); else return Yes(x+1,y-1) + 2; }</pre>

Question 28 In the code to the right, what is output on line #1? A) 10 B) 12 C) 14 D) 16 E) 18	<pre> TreeMap<Character,Integer> Cup; Cup = new TreeMap<Character,Integer>(); </pre>
Question 29 In the code to the right, what is output on line #2? A) 10 B) 12 C) 14 D) 16 E) 18	<pre> Stack<Integer> Bowl; Bowl = new Stack<Integer>(); Cup.put('D',18); Cup.put('A',12); Cup.put('C',14); Cup.put('B',10); Cup.put('C',16); </pre>
Question 30 In the code to the right, what is output on line #3? A) [16, 18] B) [18, 12] C) [12, 16] D) [18, 14] E) [18, 16]	<pre> char x; for(x='D';x>='A';x--) Bowl.push(Cup.get(x)); out.println(Bowl.pop()); // Line 1 Bowl.pop(); out.println(Cup.get('C'));// Line 2 out.println(Bowl); // Line 3 </pre>
Question 31 What is the output of the code segment shown on the right? A) false false B) false true C) true false D) true true E) no output - there is a compile error	<pre> boolean A = false; boolean B = true; for (int x = 1; x<=12; x++) if(x*x%2==0) A = B; else B = !B; out.println(A + " " + B); </pre>

Question 32 In the code to the right, what is output by line #1? A) 5 B) 6 C) 12 D) 14 E) 16	
Question 33 In the code to the right, what is output by line #2? A) 12 B) 24 C) 28 D) 36 E) 42	<pre> public class Soccer { private int A; private int B; private int C; public Soccer(int x, int y, int z) { A = x; B = y; C = z; } </pre>
Question 34 In the code to the right, what is output by line #3? A) 20 B) 40 C) 54 D) 60 E) 72	<pre> public Soccer(int x, int y) { this(y, x, x+y); } public Soccer(int x) { this(x, 2*x); } public Soccer() { this(10); } public int primetime() { return A+B+C; } } //client code Soccer R = new Soccer(1,2,3); out.println(R.primetime()); // #1 Soccer S = new Soccer(4,8); out.println(S.primetime()); // #2 Soccer T = new Soccer(); out.println(T.primetime()); // #3 </pre>

Question 35 What is the output of the code segment shown on the right? A) 0 B) 1 C) 7 D) 17 E) 64	<pre>int B = 127; B = B >> 5; B = B << 4; B = B + 8; B = B >> 3; out.println(B);</pre>
Question 36 What is the output of the code segment shown on the right? A) UNI B) NIV C) NI D) SIT E) SI	<pre>String St = "UNIVERSITY"; int L = St.indexOf("I"); out.print(St.substring(L-1,L+1));</pre>
Question 37 What is the output of the code segment shown on the right? A) 44 66 11 33 22 B) 44 44 44 44 44 C) 22 22 22 22 22 D) 22 33 11 66 44 E) 44 66 11 66 44	<pre>int[] Pogo = {22,33,11,66,44}; int x = 10; int y = 4; for(x = 0;x<=4;x++) { Pogo[x]=Pogo[y]; Pogo[y]=Pogo[x]; y--; } for(int i = 0; i<=4; i++) out.print(Pogo[i]+" ");</pre>
Question 38 What is the output of the code segment shown on the right? A) 5 B) 50 C) 65 D) 500 E) 650	<pre>int A = 50; int B = 65; int F = -1; for (int x=1; x<=A; x++) if (A%x==0 && B%x==0) F = x; out.print(A * B / F);</pre>
Question 39 Evaluate the prefix expression to the right. Write your answer in the answer blank for #39.	+ * / - 90 20 10 3 7
Question 40 To the right, we have begun to list all the 6-digit binary numbers from 000000 to 111111. So far, we have listed only 8 of them. If we were to list all 64 of these 6-digit binary numbers, how many "ones" would be written? So far, we have written 12 "ones"	000000, 000001, 000010, 000011 000100, 000101, 000110, 000111

Explanations:

1.	C	Convert all to Base 16 and then add. $10010111_2 = 97_{16}$ $10100011_2 = A3_{16}$ $7 + 3 = 10$ which would be an A $9 + A(10) = 19$ which is 13 (3 carry the 1) $97_{16} + A3_{16} = 13A_{16}$
2.	D	Use order of operations. Perform integer modulus first. $1 + 29 \% 10 / 4 + 3$ $1 + 9 / 4 + 3$ Next do integer division. $1 + 2 + 3$ Now add left to right. $1 + 2 + 3 = 6$
3.	A	Since there are only print statements, new lines will only be invoked at the <code>\n</code> new line escape character. So new lines will occur after Blue, Red, and Yellow
4.	E	<code>str.indexOf("c",2)</code> This will find the position of "c" if you start looking at position 2. Therefore it "misses" the first "c" and finds the "c" in position 9
5.	B	A is true. B is "not A" meaning B is false. Now. $!A \ \&\& \ (A \ \ B) \ ^ \ B$ First, substitute all the values $!true \ \&\& \ (true \ \ false) \ ^ \ false$ Next, evaluate <code>!true</code> $false \ \&\& \ (true \ \ false) \ ^ \ false$ Now, do the parenthesis $false \ \&\& \ true \ ^ \ false$ Next is "xor" $false \ \&\& \ true$ Using and false is the final answer
6.	A	<code>Math.sqrt(31)</code> returns a value greater than 5.0 but less than 6.0 <code>Math.floor()</code> will round that value down to 5.0 <code>(int)</code> type casts that value as an integer 5
7.	C	Even though A is a double, the evaluation begins with integer operations. $8 * 4$ is 32, then $32 / 10$ is 3 using integer division. Next add 1 (another integer) to get 4 before a double is finally introduced giving us 6.2
8.	E	$20 \% 7$ is 6 So case "6" is invoked. Because there are no break statements, the output will begin with F and keep doing all the inputs through the end of the switch statement.
9.	C	$x = x / 2 + 1$ The values of x in the iterations would be: 50 26 14 8 5 3 then the value 2 would stop the process, but not print a 7th x.
10.	A	The two arrays are the same length, so the else is invoked. Remember the first element in an array has an index of 0. <code>four[5]</code> is 24 <code>five[4]</code> is 25 The sum is 49

11.	D	St = "12 24 36 48"; The Scanner go accesses the String St. The first two go.nextInt() calls treat the inputs as integers and thus adds 12 + 24 to get 36 The next two go.next() calls treat the inputs as Strings and thus adds "36"+"48" = "3648" So, we get 363648 as the output
12.	B	The loop itself runs through the values 1 2 4 8 16 32. The accumulator adds these values one by one. That gets a total of 63 But with every iteration, 10 more is added, for a total of 60. The output is 63 + 60 = 123
13.	E	a = 15, b = 12, c = 9 ; a=1111 b=1100 c=1010 c ^ b a - c & b ^ a 9 ^ 12 15 - 9 & 12 ^ 15 9 ^ 12 15 - 9 & 12 ^ 15 (first, subtract) 9 ^ 12 6 & 12 ^ 15 (now it is bit-wise time) 1001 ^ 1100 0110 & 1100 ^ 1111 1001 ^ 1100 0110 & 1100 ^ 1111 (do and) 1001 ^ 1100 0100 ^ 1111 1001 ^ 1100 0100 ^ 1111 (do leftmost xor) 0101 0100 ^ 1111 0101 0100 ^ 1111 (do other xor) 0101 1011 (finish with the or) 1111 = 15
14.	C	Byte values range from -128 to 127 Byte_MIN_VALUE = -128 -128 + 127 = -1
15.	D	Here is the progression of messi. [] [10] [10,12] [10,12,12] [10,12,12,14] [12,12,14] [12,12,14,16] [12,12,14,16,10] [10,12,12,14,16,10]
16.	A	In each iteration, the string loses the first three letters, but that first letter is saved as it moves to the end. There are 4 iterations. (0) ABCDEFGHIJ (1) DEFGHIJA (2) GHIJAD (3) JADG (4) GJ
17.	B	This is a Fibonacci algorithm starting with a 3 and a 5. Each new element is the sum of the previous two elements. 3 5 8 13 21 34 55 89 144 233 377 The code prints element #8
18.	E	The loop iterates 3 times. For A, B, then C. A: St becomes "AA" B: St becomes AABAAB C: St becomes AABAABCAABAABC The length is 14

19.	D	[5,1,2,9,2,6,7,4,1,7] = original array Loop goes from 1 to 8. Watch the array change. 1 - [5,3,2,9,2,6,7,4,1,7] 2 - [5,3,6,9,2,6,7,4,1,7] 3 - [5,3,6,4,2,6,7,4,1,7] 4 - [5,3,6,4,5,6,7,4,1,7] 5 - [5,3,6,4,5,6,7,4,1,7] 6 - [5,3,6,4,5,6,5,4,1,7] 7 - [5,3,6,4,5,6,5,3,1,7] 8 - [5,3,6,4,5,6,5,3,5,7] Element #5 is a 6
20.	D	This is a portion of a sort routine called the "flag-tag" or "Supreme Court" sort. The elements in the list array represent the indices of the where each corresponding cool item should be in a sorted array. 17 is greater than 3 other values in the cool: 12, 8, and 3
21.	A	3 is greater than 0 other values in cool
22.	D	17 is larger than 7 items 12,8,3,1,11,5,6
23.	E	This adds the ASCII codes of 'A', 'B', and 'C' It is nice to know that the ASCII Code for 'A' is 65 and 'a' is 97. Here we add 65+66+67 = 198
24.	A	The A loop iterates 10 times. For each iteration of the A loop, the B loop iterates 11 times. For each iteration of the B loop, the C loop iterates 7 times. $10 * 11 * 7 = 770$
25.	D	Yes(5,2) goes straight to the stopping state condition. $5 * 2 = 10$
26.	B	Yes (3,3) first goes to the middle option. Yes(3,3) = Yes(6,4) Yes(6,4) = 24
27.	A	Yes(0,7) = Yes(1,6) + 2 Yes(1,6) = Yes(2,5) + 2 Yes(2,5) = Yes(3,4) + 2 Yes(3,4) = Yes(4,3) + 2 Yes(4,3) = 12 $12 + 2 + 2 + 2 + 2 = 20$
28.	B	After the 5 Cup.put() lines, we have: 'A' = 12 'B'=10 'C'=16 'D' = 18 These are pushed onto the Bowl stack in reverse order with 18 being on the bottom. At this point the stack is [18,16,10,12] The first pop removes the 12
29.	D	Continuing from #28 Cup.get('C') gives us 16
30.	E	Continuing from #28 Cup is now [18,16,10] 10 is popped. Cup is now [18,16]

31.	D	The loop will test the numbers 1,4,9,16,25,36,49,64,81,100,121,144 Notice that the alternate from odd to even A = false, B=true. if even, A takes on B's value if odd, B's value flips 1 odd - now B=false 4 even - now A=false 9 odd - now B=true 16 even - now A=true 25 odd - now B=false 36 even - now A=false 49 odd - now B=true 64 even - now A=true 81 odd - now B=false 100 even - now A=false 121 odd - now B=true 144 even - now A=true
32.	B	R utilizes the 3-parameter constructor sending in 1,2,3 A=1 B=2 C=3 primetime returns $1+2+3 = 6$
33.	B	S utilizes the 2-parameter constructor sending in 4,8 It then calls the 3-parameter constructor sending in 8,4,12 A=8 B=4 C=12 primetime returns $8+4+12 = 24$
34.	D	T utilizes the 0-parameter constructor It then calls the 1-parameter constructor sending in 10 Then it calls the 2-parameter constructor sending in 10,20 It then calls the 3-parameter constructor sending in 20,10,30 A=20 B=10 C=30 primetime returns $20+10+30 = 60$
35.	C	B = 127 (1111111) Each right shift >> divides by 2 using integer division Each left shift << multiplies by 2 B >> 5 (right shift 5) divides $127/32 = 3$ (11) B << 4 (left shift 4) multiplies $3 * 16 = 48$ (110000) B + 8 Now add 8 $48+8 = 56$ (111000) B >>3 (right shift 3) divides by 8 $56/8 = 7$ (111)
36.	C	St = "UNIVERSITY" L will have a value of 2, the location of the first "I" St.substring(1,3) is "NI"
37.	E	The original values are [22,33,11,66,44] The loop iterates 5 times. Pogo[0] becomes Pogo[4] [44,33,11,66,44] The next line has no effect Pogo[4] = Pogo[0] since they are already equal. The second iteration makes Pogo[1] = Pogo[3] [22,66,11,66,44] Again the next line has no effect. In fact, no other changes are made because: Pogo[2] = Pogo[2] causes no change. Pogo[3] = Pogo[1] causes no change. Pogo[4] = Pogo[0] causes no change.
38.	E	The loop is designed to find the greatest common factor of 50 and 65 Thus $F = 5$ Now doing $50 * 65 / 5$ you get 650 This is a good way to get the least common multiple since $A * B = GCF(A,B) * LCM(A,B)$

39.	28	+ * / - 90 20 10 3 7 + * / 70 10 3 7 + * / 70 10 3 7 + * 7 3 7 + 21 7 28
40.	192	If we listed all 64 binary numbers from 000000 to 111111, we would be listing 384 digits (64*6). If it is a complete list, there would be half zeros and half ones. Thus, we would have 192 of each.

★ ANSWER KEY – CONFIDENTIAL ★

UIL COMPUTER SCIENCE – 2023 INVITATIONAL B

Questions (+6 points for each correct answer, -2 points for each incorrect answer)

1) C	11) D	21) A	31) D
2) D	12) B	22) D	32) B
3) A	13) E	23) E	33) B
4) E	14) C	24) A	34) D
5) B	15) D	25) D	35) C
6) A	16) A	26) B	36) C
7) C	17) B	27) A	37) E
8) E	18) E	28) B	38) E
9) C	19) D	29) D	*39) 28
10) A	20) D	30) E	*40) 192

* See "Explanation" section below for alternate, acceptable answers.

Note: Correct responses are based on **Java SE Development Kit 17 (JDK 17)** from Oracle, Inc. All provided code segments are intended to be syntactically correct, unless otherwise stated (e.g., "error" is an answer choice) and any necessary Java SE 17 Standard Packages have been imported. Ignore any typographical errors and assume any undefined variables are defined as used.